



Nombre:

Grupo:

Pregunta 1:

- [] Dada la siguiente definición:
int **p;
p puede definir una matriz bidimensional, un vector u otro uso diferente.
- [] Aunque un dato almacenado en un vector dinámico no se borre nunca a lo largo de todo su ciclo de vida, podría cambiar su dirección de memoria.
- [] A nivel de eficiencia, es igual implementar una cola dinámica usando una lista (std::list) que un vector (std::vector).
- [] Si se introducen datos ordenados de forma descendente en un AVL, el tipo de rotación que se realizaría siempre sería el caso 4.
- [] Sabemos que la siguiente sentencia es incorrecta para un contenedor asociativo de STL:
(*it).first= &p;

- [] Si al buscar el punto más cercano al punto p en un quadtree, éste no se encontrara en el mismo nodo que p, entonces debe estar en otro de sus otros tres nodos hermanos.
- [] Si las tablas hash T1 y T2 almacenan los mismos datos y $\lambda_1=0.8$ y $\lambda_2=0.7$ son sus respectivos factores de carga, entonces sabemos que el tamaño de T1 es mayor que el de T2.
- [] Es posible indexar 24.000 datos en un árbol B de orden 32 y altura 2.
- [] Averiguar las aristas adyacentes de entrada a un vértice en un digrafo implementado mediante listas de adyacencia es $O(k)$, siendo k el número de vértices adyacentes.
- [] Se pueden listar todos los datos de un árbol ABB en orden inverso a su definición de forma recursiva en $O(n)$.

Pregunta 2: Realizar en orden las operaciones indicadas sobre la siguiente tabla de dispersión cerrada de enteros positivos, con doble dispersión por división, donde $h1(x)=x\%11$ y $h2(x)=1+(x\%7)$. Mostrar el estado de las celdas de la tabla después de cada operación. En caso de colisión al realizar una operación, y para la búsqueda, indicar las diferentes posiciones que se van comprobando en la secuencia de exploración hasta completarla.

0	1	2	3	4	5	6	7	8	9	10
		35				31	18	19	20	8

Insertar (I) y borrar (B) la siguiente secuencia de datos: I(15), I(17), B(8), buscar (24), I(26).

Pregunta 3:

1. Implementar el constructor de un árbol binario de búsqueda $ABB<T>$ a partir de un vector de STL con datos ordenados de menor a mayor de forma que quede equilibrado en altura (compatible con AVL). Se puede hacer de forma recursiva haciendo que el dato central del vector (completo o parte de éste) sea la raíz.

```
ABB<T>::ABB(const vector<T> &v);
```

2. Implementar la función de concatenación de listas simplemente enlazadas sobrecargando el operador de suma (operator+) de tal forma que se cree una nueva lista formada por una copia de los datos de (*this) y a continuación con los de la lista l que entra como parámetro.

```
ListaEnlazada<T> ListaEnlazada<T>::operator+ (const ListaEnlazada<T> &l)
```

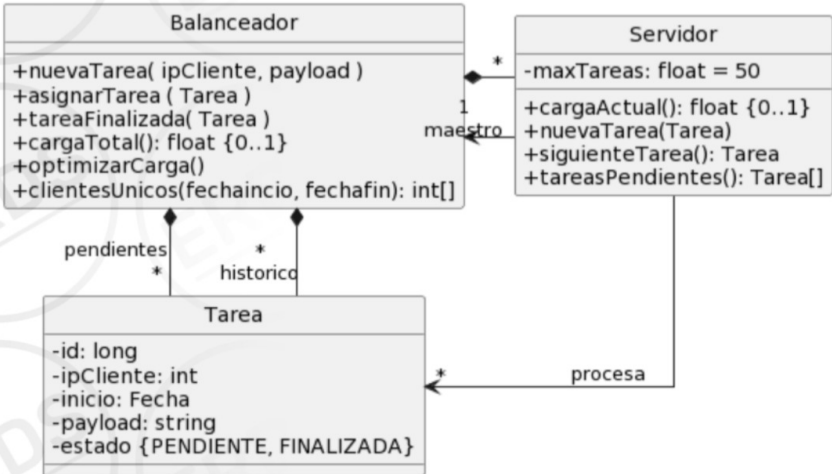
Pregunta 4:

El siguiente diseño modela parte del control de un sistema de balanceo de cargas para un reparto óptimo de tareas de ejecución en un proveedor en la nube, que puede procesar al día miles de tareas solicitadas desde clientes en Internet. Cuando se recibe una nueva tarea, se marca como pendiente y se asigna para su procesamiento a dos Servidores (cada servidor representa una máquina virtual) aprovisionados bajo demanda por el sistema (Balanceador). El Balanceador debe repartir las tareas entre los servidores existentes garantizando que la carga de trabajo de cada uno (n^0 de tareas asignadas entre máximas que soporta) sea similar y se mantenga por debajo de 1. Si no es posible, crea un nuevo servidor para ello.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

Además, si el promedio de carga de todos los servidores (carga total del balanceador) baja excesivamente, el balanceador eliminará alguno (la máquina virtual asociada se apagará) repartiendo sus tareas pendientes entre el resto para optimizar el coste total del sistema. Como la máquina virtual asociada a cada Servidor podría bloquearse o demorar sus cálculos en exceso, el balanceador trata de garantizar que las tareas siempre se procesen lo antes posible asignándoselas a dos de ellos (no importa que una tarea se finalice dos veces), aunque cuando son procesadas se marcan como finalizadas para evitar que otro servidor que las tuviera pendientes comience su procesamiento. Definir en C++ las clases *Tarea*, *Servidor* y *Balanceador* e implementar tanto los métodos indicados como cualquier otro que sea necesario. Suponer implementada la clase Fecha con las operaciones habituales vistas en prácticas.



Tarea: contiene la información que debe ser procesada (*payload*) y el estado en el que se encuentra en el sistema: pendiente de procesarse o ya finalizada. Por cuestiones administrativas debe almacenar la fecha en que se solicitó y la *ip* del cliente que lo hizo. Cada una se debe identificar internamente, y de forma unívoca, por un código numérico.

Servidor: gestiona las tareas de cálculo que una determinada máquina virtual asociada tendrá que procesar. Por defecto hay un tope de 50 tareas máximas por servidor pero en el futuro podría variar según las características de la máquina virtual utilizada. **nuevaTarea:** añade una nueva tarea al servidor de forma que se procesen en primer lugar las tareas con mayor tamaño de datos a procesar (*payload*). Al finalizar el método el servidor comenzará a procesar su tarea con más prioridad. **siguienteTarea:** este método es llamado por el propio servidor cuando ha finalizado el procesamiento de su tarea más prioritaria. Esta será eliminada de sus tareas por procesar y se notificará al *Balanceador* mediante su método *tareaFinalizada*. Se descartarán a continuación aquellas tareas más prioritarias que ya hubieran sido finalizadas (por otro servidor). Devuelve la tarea con más prioridad con la que el servidor comenzará trabajar cuando finalice el método. **cargaActual:** valor entre 0 y 1 obtenido como el número de tareas que debe procesar el servidor entre el máximo establecido para dicho servidor (atributo *maxTareas*). **tareasPendientes:** elimina y devuelve las tareas por procesar del servidor. Utilizado por el método *optimizarCarga* del balanceador antes de que elimine al servidor.

Balanceador, encargado de crear y repartir entre los servidores las nuevas tareas que se reciban. También se encarga de crear o destruir los servidores para garantizar un uso óptimo de los recursos de computación gestionados. **nuevaTarea:** crea una nueva tarea pendiente en el sistema, identificándola con un *id* único generado automáticamente, con la *fecha* actual y asignándola (método *asignarTarea*) a dos servidores para su procesamiento, garantizando así una alta disponibilidad en caso de que uno de ellos se pudiera bloquear. Los objetos *Fecha* inicializados por defecto tienen la fecha actual del sistema. **asignarTarea:** asigna una tarea a un servidor utilizando el esquema *round-robin*, es decir, al siguiente de forma consecutiva entre todos los servidores disponibles y siempre que su carga actual no sea superior a 1. Si no es posible asignar la tarea a ningún servidor, crear uno nuevo y asignársela a él. **tareaFinalizada:** método llamado desde un Servidor para notificar que una tarea ha sido finalizada. Esta será marcada como finalizada y transferida permanentemente de las tareas pendientes del balanceador al histórico. Si la tarea ya se hubiera finalizado anteriormente por otro servidor será ignorada. Si la carga total del sistema (método *cargaTotal*) es inferior a 0.5 se llamará al método *optimizarCarga*. **cargaTotal:** devuelve la carga total del sistema (valor entre 0 y 1) obtenida como la media aritmética de las cargas de los servidores existentes (suma total de las cargas de los servidores dividida por el número de servidores). **optimizarCarga:** método utilizado para evitar que los servidores estén infrautilizados. Elimina uno de ellos pero, antes de hacerlo, obtiene sus tareas pendientes (*Servidor::tareasPendientes*) y las reparte entre el resto, aumentando por consiguiente la carga total del sistema. **clientesUnicos:** consulta de forma eficiente las tareas finalizadas entre dos fechas determinadas y devuelve las direcciones *IP*, sin repetir, de los clientes que las hayan realizado.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional